Práctica 08.
Programación en Shell.
Ejecución condicional de órdenes.

Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera Copyright: Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0)

Índice

Índice **1**
Introducción **2**
La estructura condicional if - then - else 2
Condicionales y operadores **3**
Operadores para aplicar a archivos: 4
Operadores para aplicar a números: 5
Operadores para aplicar a cadenas de caracteres: 5
Condiciones compuestas 6
Comparaciones avanzadas **6**
Ejemplos del uso de condicionales en shell script **7**
Comprobar si se han pasado todos los argumentos 7
Comprobar si existe o no un archivo 8
Comparar cadenas de caracteres 8
Comparar expresiones y números 9
Ejercicios **10**
Propuestas de bash scripts **11**

Introducción

En ocasiones, un *shell script* deberá ejecutar un conjunto de órdenes si se da una condición y otro distinto si no se da. Es lo que se denomina ejecución condicional de órdenes.

Como en la mayoría de lenguajes de programación, en la programación de *shell scripts* existe la estructura `if-then-else`, que son palabras clave o reservadas del intérprete de órdenes, y que permiten programar la ejecución condicional de órdenes.

En esta sesión de prácticas se estudia esta estructura y cómo se evalúan las expresiones lógicas que nos permitirán construir programas más flexibles y útiles.

En Ubuntu 26.04 LTS. Ubuntu 26.04 LTS incluye bash 5.3, y Ubuntu 24.04 LTS, bash 5.2. Todos los ejemplos de este guion se comportan igual en las dos versiones; solo cambia la redacción de algún mensaje de error. Los ejemplos se han obtenido en Ubuntu 24.04 LTS, la versión del aula.

La estructura condicional `if - then - else`

Si se escribe en un terminal `type -a if`, el terminal responde:

```
alumno@so-lab01: ~
alumno@so-lab01:~$ type -a if
if es una palabra clave del intérprete de órdenes
```

Es decir, `if` no es una orden, sino que es una palabra clave que indica al intérprete de órdenes que debe interpretar lo que sigue al `if` como un condicional. La sintaxis correcta para escribir un condicional en *shell script* es:

```
if orden_o_condición
then
    órdenes si la condición es verdadera (código de terminación 0)
elif otra_condición          # opcional; puede repetirse
then
    órdenes si la otra condición es verdadera
else                         # opcional
    órdenes en cualquier otro caso
fi
```

Si la orden de la condición termina con código 0 (verdadero) se ejecuta la parte `then`, y si termina con cualquier otro código (falso), la parte else. Es el criterio contrario al de C o Java, donde el cero es falso.

Los corchetes `[ ]` y los espacios que hay entre ellos y la condición son obligatorios: `[` es una orden, y lo que escribimos dentro, sus argumentos. Es un fallo muy común no poner espacio entre el primer corchete y el primer carácter de la condición. Por ejemplo, si el primer corchete va pegado a `-d`, el intérprete busca una orden llamada «`[-d`» y no la encuentra:

```
alumno@so-lab01: ~
alumno@so-lab01:~$ if [-d ${HOME}/bin]; then echo existe; fi
[-d: no se encontró la orden
```

También es frecuente olvidar el espacio entre el final de la condición y el último corchete `]`. Por ejemplo, si el último corchete va pegado a `bin`, la orden `[` no recibe su último argumento, `]`, y lo indica. Con los espacios, la orden funciona:

```
alumno@so-lab01: ~
alumno@so-lab01:~$ if [ -d ${HOME}/bin]; then echo existe; fi
bash: [: falta un `]'
alumno@so-lab01:~$ if [ -d ${HOME}/bin ]; then echo existe; fi
existe
```

Las estructuras condicionales en *shell script* comienzan con la palabra reservada `if` y terminan con la palabra reservada `fi`.

Condicionales y operadores

Para comparar en Bash se utiliza `test` o `[`. Ambos son totalmente equivalentes, y ambos están implementados, en el propio Bash. En el caso `[`, por cuestiones prácticas es obligatorio también utilizar su pareja `]` o de otra forma genera un error. Esta es la razón para que tenga que estar separado el corchete de la expresión que viene a continuación (**debe incluirse un espacio en blanco**), porque es un comando.

Para construir las condiciones, se usan valores, sustituciones de parámetros y operadores.

Por ejemplo:

```
[ -r "${1}" ]
```

es una condición sintácticamente bien construida que comprueba si el valor del parámetro posicional 1, es el nombre de un archivo que existe y tiene permisos de lectura (*readable*). En la programación shell, existen muchos operadores que podemos utilizar para construir nuestras condiciones.

Operadores para aplicar a archivos:

Los nueve primeros son unarios, es decir sólo necesitan un operando que se escribe a continuación del operador:

```
[ operador archivo ]
```

Los dos últimos son binarios, es decir, requieren dos operandos que se escriben uno a cada lado del operador:

```
[ archivo1 operador archivo2 ]
```

`-r` ***Readable*** (Lectura). Comprueba si lo que va a continuación es el nombre de un archivo que existe y tiene permisos de lectura.

`-x` **eXecutable** (Ejecutable). Comprueba si el archivo existe y es ejecutable.

`-h` Comprueba si el archivo existe y es un enlace simbólico[1]. También se escribe `-L` ***Symbolic Link*** (Enlace simbólico). Con un enlace roto, `-h` es verdadero y `-e` es falso.

`-f` ***regular File*** (Archivo regular). Comprueba si el archivo existe y es un fichero regular u ordinario (no es un directorio, ni otro tipo de archivo especial)

`-d` **Directory** (Directorio). Comprueba si el archivo existe y es un directorio.

`-s` ***Size*** (Tamaño). Comprueba si el archivo existe y no está vacío (su tamaño es mayor que cero).

`-e` ***Exists*** (Existe). Comprueba si el archivo existe.

`-O` ***Owned by owner*** (Propiedad del dueño). Comprueba si el archivo es propiedad de quien ejecuta el *shell script*. `O` es la letra o mayúscula, no un cero.

`-G` ***Group owned*** (Propiedad del grupo). Comprueba si el archivo pertenece al grupo de quien ejecuta el *shell script*.

`-nt` ***Newer Than*** (Más nuevo que). Se coloca entre dos nombres de archivos y devuelve verdadero si el primero tiene fecha de modificación más reciente que el segundo.

`-ot` ***Older Than*** (Más viejo que). Comprueba si el primer archivo tiene fecha de modificación más antigua que el segundo.

[1] Cuando se crearon las primeras herramientas de diagnóstico en los años 80, la letra l (link) ya se usaba en otros comandos o contextos. Para diferenciar los tipos de enlaces, se asoció de manera abstracta la h a los conceptos de la familia de enlaces (links), jugando con la nomenclatura de Hard/Soft.

Operadores para aplicar a números:

Todos ellos son binarios. Es decir, se aplican a dos argumentos que se escriben uno a cada lado del operador.

`-eq` ***Equal to*** (Igual a). Comprueba si los dos números que se pasan como argumentos son iguales.

`-ne` ***Not equal to*** (No igual a / Distinto de). Comprueba si los dos números son distintos.

`-ge` ***Greater than or equal to*** (Mayor o igual que). Comprueba si el primer número es mayor o igual que el segundo.

`-gt` ***Greater than*** (Mayor que). Comprueba si el primer número es mayor estricto que el segundo.

`-le` ***Less than or equal to*** (Menor o igual que). Comprueba si el primer número es menor o igual que el segundo.

`-lt` ***Less than (Menor que)***. Comprueba si el primer número es menor estricto que el segundo.

Operadores para aplicar a cadenas de caracteres:

Los dos primeros son unarios y los cuatro últimos, binarios. Los dos últimos (`<` y `>`) son también los símbolos de la redirección: dentro de `[ ]` hay que escribirlos con una barra invertida (`\<` y `\>`); dentro de `[[ ]]` se escriben sin ella. Es conveniente recordar que debe haber espacios en blanco a la izquierda y derecha de cada operador.

`-z` ***Zero*** (Cero). Indica si la longitud de la cadena que se pasa como argumento es cero (cadena vacía).

`-n` ***Non-zero o Not empty*** (No cero / No vacía). Indica si la longitud de la cadena es mayor que cero.

`=` Comprueba si las dos cadenas de caracteres son iguales.

`!=` Comprueba si las dos cadenas de caracteres son distintas.

`<` Indica si la primera cadena es menor que la segunda. Hace una comparación lexicográfica (es decir, **comparando carácter a carácter**). Por ejemplo, la cadena “adiós” es menor que la cadena “hola”. El orden depende de la configuración regional: en `es_ES.UTF-8`, «Zorro» va detrás de «abeja»; con `LC_ALL=C`, delante, porque las mayúsculas se ordenan antes.

`>` Indica si la primera cadena es mayor que la segunda.

Condiciones compuestas

Se pueden usar condiciones compuestas, por ejemplo comprobar a la vez si un archivo es legible y ejecutable. Para ello se usan los operadores lógicos `&&` (*and*) y `||` (*or*) para separar dos condiciones simples.

```
[ -r "${file}" ] && [ -x "${file}" ]
```

También se puede usar el operador unario `!` para negar el resultado de una condición.

Importante:

Como en cualquier lenguaje de programación, si en un *shell script* no se escribe la sintaxis de las órdenes correctamente se producen errores.

Para aprender a programar *shell scripts* es necesario estudiar la sintaxis y practicarla con diversos ejemplos hasta que la aprendas.

Comparaciones avanzadas

También se pueden utilizar dobles corchetes `[[` para realizar tus comparaciones. Los dobles corchetes pueden resultar ser una mejora respecto a los simples en ciertas situaciones. Las diferencias entre uno y otro son las siguientes:

- No tienes que utilizar las comillas con las variables, los dobles corchetes trabajan perfectamente con los espacios.
  Así `[ -f "${file}" ]` es equivalente a `[[ -f ${file} ]]`.
- Con `[[` puedes utilizar los operadores `||` y `&&`, así como < y > para las comparaciones de cadena.
- Puedes utilizar el operador =~ para expresiones regulares[2],
  como por ejemplo `[[ ${respuesta} =~ ^s(i)?$ ]]`
- También puedes utilizar comodines como[3]
  por ejemplo en la expresión `[[ abc == a* ]]`

[2] `[[ $respuesta =~ ^s(i)?$ ]]` verifica si la variable `$respuesta` contiene **exactamente** `"s"` o `"si"` (sin ningún carácter adicional antes o después). Esta expresión es útil, por ejemplo, para confirmar si un usuario respondió afirmativamente con `"s"` o `"si"` pero no coincide con: `"sí"` (acentuada), `" s "` (con espacios), `"no"`, `"siempre"`. **=~**: El operador que compara la cadena en `$respuesta` con la expresión regular proporcionada. `^` inicio de la línea, `$` final de la línea. `(i)?` es una subexpresión opcional, `?` indica que la `i` es opcional.

[3] `[[ abc == a* ]]` devolvería `true` porque `a*` en este caso significa "a seguido de cualquier número de caracteres"

Es posible que te preguntes por la razón para seguir utilizando `[` simple corchete en lugar de doble. La razón es la compatibilidad: `[[` es una extensión de bash, que no existe en sh. En Ubuntu, sh es `dash`, y un guion con `#!/bin/sh` que use `[[` termina con «`[[: not found`». Si el guion empieza por `#!/bin/bash`, puedes usar `[[` sin problemas.

Ejemplos del uso de condicionales en *shell script*

Comprobar si se han pasado todos los argumentos.

Uno de los usos de los condicionales en programas *shell script* es comprobar si se han pasado todos los argumentos al programa. En aquellos programas que requieran argumentos es muy importante que se compruebe que se han pasado los parámetros posicionales adecuados, ya que un parámetro que no existe siempre se sustituye por la cadena vacía y esto podría causar errores en el programa.

Ejemplo: Escribir un *shell script* que reciba dos argumentos y comprobar si se han pasado justamente dos argumentos.

```
#!/bin/bash
# Autor: Francisco de Asís Conde Rodríguez
# Descripción: Comprueba si al script se le han pasado
#              justamente dos argumentos.

if [ ! ${#} -eq 2 ]          # (1)
then                         # (2)
    echo "Error, se han pasado ${#} argumento(s)"     # (3)
fi
```

(1) Como sabemos de la sesión anterior, el parámetro especial `#` contiene el número de parámetros posicionales que se han pasado al *shell script* cuando se ejecutó. Por tanto si comprobamos si es distinto de `2`, habremos comprobado si a nuestro script se le han pasado, o no, justamente dos argumentos.

(2) La palabra reservada `then` va en la línea siguiente a la del `if`, o en la misma línea separada por un punto y coma: `if [ ${#} -ne 2 ]; then`. Sin el punto y coma se produce un error de sintaxis.

(3) El sangrado es opcional, pero si se incluye, el código es mucho más legible.

Comprobar si existe o no un archivo

Como administrador del sistema operativo, en muchas ocasiones tendrás que hacer comprobaciones sobre archivos, qué permisos tienen o a quién pertenecen, y en función de esas comprobaciones realizar unas tareas de administración u otras.

Ejemplo: Escribir un *shell script* que reciba un argumento (el nombre de un archivo ejecutable) y compruebe si ese archivo está en el directorio `bin` del usuario que ejecuta el *shell script* y realmente es ejecutable. Si no está debe escribir por pantalla que no existe ese archivo, y si no es ejecutable debe escribir por pantalla que no es ejecutable.

```
#!/bin/bash
# Autor: Francisco de Asís Conde Rodríguez
# Descripción: Comprueba si un nombre que se pasa como
#              argumento corresponde a un programa
#              ejecutable situado en el directorio ~/bin

if [ ${#} -ne 1 ]
then
    echo "Error. Se debe pasar un argumento"
    exit 1
fi

if [ ! -e "${HOME}/bin/${1}" ]     # (1)
then
    echo "No existe el script: ${1}"
else
    if [ ! -x "${HOME}/bin/${1}" ]
    then
        echo "${1} no es un ejecutable"
    fi
fi
```

(1) La variable `HOME` contiene la ruta completa del directorio base del usuario que ejecuta el script. Las llaves son necesarias cuando tras el nombre viene un carácter que podría formar parte de él (una letra, un dígito o _): `${v}_x`. Aquí no lo son, porque la barra no puede formar parte de un nombre; `$HOME/bin/$1` funciona igual, aunque las llaves no sobran. Las comillas, en cambio, sí son necesarias: sin ellas, un nombre con espacios rompe la condición.

Comparar cadenas de caracteres.

En *shell script*, todos los valores se tratan como cadenas de caracteres a menos que se indique lo contrario, por tanto, es muy importante saber escribir condicionales en los que los argumentos de las condiciones sean cadenas.

Ejemplo: Escribir un *shell script* que reciba exactamente dos argumentos y que compruebe que son distintos. Si no se escriben dos argumentos, o si estos son iguales debe escribir un error en la pantalla.

```
#!/bin/bash
# Autor: Francisco de Asís Conde Rodríguez
# Descripción: Comprueba si los dos argumentos que se dan
#              son iguales.

if [ ${#} -ne 2 ]
then
    echo "Error. No se han dado dos argumentos"
else
    if [ "${1}" = "${2}" ]
    then
        echo "Error. Los argumentos son iguales"
    fi
fi
```

La comprobación de la igualdad de argumentos es útil por ejemplo a la hora de renombrar un archivo. En este caso, si el nombre del archivo que se renombra y el nuevo nombre que se quiere dar al archivo son iguales, no se debería ejecutar la orden de renombrar.

Comparar expresiones y números.

En este script se comprueba el número de argumentos y que los argumentos contengan sólo números.

Ejemplo: Script *`triangulo.sh`* al que se pasan 3 argumentos y devuelve el tipo de triángulo utilizando las siguientes condiciones:

Escaleno: Un triángulo en el que cada lado tiene una longitud diferente.
Isósceles: Un triángulo en el que 2 de sus lados tienen la misma longitud.
Equilátero: Un triángulo en el que todos los lados tienen la misma longitud.

Para saber si una variable es numérica en Shell Script podemos comprobar con una expresión regular si todos sus caracteres son numéricos.

```
[[ "${variable}" =~ ^[0-9]+$ ]]
```

Donde indicamos que entre el comienzo (`^`) y el final (`$`) hay una o más veces (`+`) un elemento comprendido en el rango de 0 a 9 (`[0-9]`). Si hacemos esta comprobación en un `if`, nos devolverá verdadero si todos los caracteres son numéricos y falso si alguno no lo es.

```
#!/bin/bash
# Autor/a: Lina García Cabrera
# Descripción: Tipo de triángulo:
#   EQUILÁTERO, ISÓSCELES o ESCALENO.
#   Se le pasan 3 números, los lados del triángulo y dice el
#   tipo de triángulo

# Comprobar el número de argumentos
if [ ${#} -ne 3 ]
then
   echo "No has pasado 3 argumentos."
   echo "Sintaxis: ${0} lado1 lado2 lado3"
else
  # Comprobar que los argumentos son números
  if [[ "${1}" =~ ^[0-9]+$ && "${2}" =~ ^[0-9]+$ &&
        "${3}" =~ ^[0-9]+$ ]]
  then
     # Todos los lados son iguales
     if [ ${1} -eq ${2} ] && [ ${2} -eq ${3} ] && [ ${1} -eq ${3} ]
     then
       echo "El triángulo es EQUILÁTERO"
     # 2 lados son iguales
     elif [ ${1} -eq ${2} ] || [ ${2} -eq ${3} ] || [ ${1} -eq ${3} ]
     then
       echo "El triángulo es ISÓSCELES"
     else
       echo "El triángulo es ESCALENO"
     fi
  else
     echo "Alguno de los argumentos no es un número"
     echo "Sintaxis: ${0} lado1 lado2 lado3"
  fi
fi
```

Ejercicios

1. Reescribe el *shell script*:

   ```
   #!/bin/bash
   # Autor: Francisco de Asís Conde Rodríguez
   # Descripción: Comprueba si al script se le han pasado
   #              justamente dos argumentos.

   if [ ! ${#} -eq 2 ]
   then
       echo "Error, se han pasado ${#} argumento(s)"
   fi
   ```

   para que realice la misma comprobación, pero usando otros operadores.

2. Escribe un *shell script* que compruebe si se le han pasado entre dos y cuatro argumentos. En caso de que no se le hayan pasado ese número de argumentos que escriba un error, y en caso de que sí se le hayan pasado que los escriba por pantalla.

3. Escribe un *shell script* que reciba exactamente un argumento que es el nombre de un directorio. A continuación compruebe si ya existe, y si no existe que lo cree. Por último, tanto si existía previamente como si no, que le dé permisos `drwx------`

4. Escribe un *shell script* que reciba exactamente un argumento, un nombre de usuario, y compruebe si en el directorio base de ese usuario existe un archivo llamado `.profile`. Si no existe, que lo copie del directorio `/etc/skel` en el directorio del usuario y le dé permisos `-rw-r--r--`.
   Nota: Dado que el directorio de otro usuario y su `.profile` solo pueden ser creados o modificados por `root`, el guion tendrá que ejecutarse con `sudo`. Tras copiar el fichero, es necesario asignarle la propiedad del usuario mediante `chown` para que este pueda modificarlo.

5. Escribe un *shell script* que reciba exactamente dos argumentos que sean cadenas de caracteres y diga si la primera está ordenada alfabéticamente con respecto a la segunda o no.

6. La tarea de escribir un nuevo *shell script* requiere varios pasos: comprobar si el nombre del script ya existe, escribir el archivo con un shebang por defecto, darle permisos de ejecución y copiarlo al directorio `~/bin`. Escribe un *shell script* que reciba como argumento el nombre de un nuevo *shell script* que se quiere escribir y simplifique todos esos pasos.
   NOTA: En el script puedes comprobar si existe alguna orden o script con el nombre pasado como parámetro con type redirigiendo salida y error a `/dev/null`. Luego puedes usar el parámetro especial `$?`, que contiene el código de terminación de la última orden (0 si terminó sin error) para informar que ese nombre ya existe.

Propuestas de bash scripts

1. Escribe un script bash al que se le pasan 2 argumentos al que llamaremos **`cpalabra.sh`**: el nombre de un fichero de texto y una palabra o frase que debe pasarse entre comillas dobles. El script debe comprobar el número de argumentos y que el primer argumento es un fichero y el segundo un texto. El programa **cuenta el número de ocurrencias de la palabra o frase proporcionada que hay en el fichero de texto** que se pasa como argumento. Indicaciones: utiliza las órdenes `grep` (busca la opción que solo muestra las coincidencias, no la línea completa) y `wc`.

2. El comando **`du`** sirve para verificar el uso de espacio en disco en un directorio. Escriba una orden llamada **`queocupa.sh`** al que se pasan 3 argumentos: un directorio, una extensión (algo como .jpg, .png) y un número `num`. El script **muestra los `num` ficheros que más ocupan con esa extensión**. Indicaciones: Utiliza la opción de la orden `du` que muestra el espacio en Byte, Kilobyte, Megabyte, Gigabyte, Terabyte y Petabyte, usa las órdenes `grep`, `sort` (busca la opción que ordena los tamaños tal como los escribe du -h, con sus sufijos K, M, G…). El script debe comprobar el número de argumentos, que el primero es un directorio, el segundo una cadena que comienza por "." y que el tercero es un número.

3. La orden **`uniq`** sin argumentos elimina las líneas de texto que están duplicadas sólo si son adyacentes. Escribe una nueva orden que se llamará **`uniq_all.sh`** que recibe dos parámetros, 2 nombres de ficheros. Lo que hace es **quitar las líneas repetidas de un fichero** estén o no consecutivas y lo guarda en el otro fichero. El programa debe primero comprobar que se le pasan 2 argumentos, que son ficheros ordinarios y que tienen permiso de lectura. A continuación tienes un ejemplo del contenido de un fichero con líneas repetidas.

Ejemplo de fichero de entrada	Fichero resultado

`Julio Lorenzo`
`Pedro Andión`
`Celia Fernández`
`Celia Fernández`
`Juan Fernández`
`Enrique Peña`
`Julio Lorenzo`
`Pedro Andión`
`Celia Fernández`
`Juan Fernández`
`Enrique Peña`
`Julio Lorenzo`
	`Julio Lorenzo`
	`Pedro Andión`
	`Celia Fernández`
	`Juan Fernández`
	`Enrique Peña`

4. Escriba una orden llamada **`masenlaces.sh`** a la que se le pasan 2 argumentos: un directorio y un número. Esta orden **muestra los *n* ficheros o directorios con más enlaces duros de ese directorio**. Indicaciones: `tail -n +2`: Omite la primera línea de la salida, que suele ser el encabezado en el caso de `ls -l` (normalmente "total XX"). La opción `-n` especifica el número de líneas que quieres mostrar, pero el signo `+` cambia el comportamiento de tail. En lugar de mostrar las últimas *`n`* líneas, `+2` indica que se deben mostrar todas las líneas comenzando desde la segunda línea. El script debe comprobar el número de argumentos, que el primero es un directorio y el segundo un número.

5. Escriba una orden llamada **`num_duplicados.sh`** a la que se pasan 2 argumentos: un número y un fichero. Esta orden **busca en un fichero de configuración** (por ejemplo, `/etc/passwd` o `/etc/group`) **si uno de sus campos numéricos tiene números duplicados**. El argumento número hace referencia al campo que debe verificar. El script debe comprobar el número de argumentos, que el primero es un número y el segundo un fichero y que se puede leer. Por ejemplo, en el caso de `/etc/passwd`, en el campo 3 están los UIDs, en el caso de `/etc/group`, en el campo 3 están los GIDs. Formas de invocarla sería:

   ```
   ./num_duplicados.sh 3 /etc/group
   ./num_duplicados.sh 3 /etc/passwd
   ```

   Se puede simular en un fichero para probarlo en el que se repitan números o copiar parte de la información del fichero `group` o `passwd` repitiendo algunas líneas. **Sugerencia:** consulte el comando **cut** para una primera aproximación a trocear cadenas.